Name of the Student	MADDIRALA BALAMURALI KRISHNA
Reg. No	192325015
GitHub Link	https://github.com/krishnalsk/MLA0403-DEEP-LEARNING.git

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME:DEEP LEARNING **COURSE CODE:MLA0403**

COURSE OUTCOME COVERED

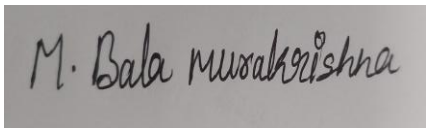
CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications.
(BTL-6)
CO Weightage: 25%

Duration: 90 minutes
Marks: 25

Assessment Tool Weightage: 40% ***Code***

of Conduct:
I MADDIRALA BALAMURALI KRISHNA (192325015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

MADDIRALA BALAMURALI KRISHNA

Name of the student:

Q1. CNN vs Fully Connected Neural Network (5 Marks)

Context: A medical imaging organization wants to classify X-ray images. Below is a comparison of a CNN against a traditional fully connected neural network (FCNN/MLP) for this task.

Parameter	Fully Connected Neural Network (FCNN)	Convolutional Neural Network (CNN)
Architectural design	Every neuron connects to every neuron in the next layer; no spatial structure preserved	Local receptive fields with convolutional filters; spatial structure of the image is preserved across layers
Motivation	Designed for generic vector inputs (tabular/flattened data)	Designed specifically for grid-like data (images), inspired by the visual cortex's local processing
Spatial feature extraction	Image must be flattened into a 1-D vector, so spatial relationships between neighbouring pixels are lost	Filters scan local neighbourhoods, preserving and exploiting spatial relationships (edges, textures, shapes)
Number of parameters	Extremely high — e.g., a 224×224×1 image to a 100-neuron layer needs ~5 million weights	Much lower due to small shared filters (e.g., a 3×3 filter = 9 weights) reused across the whole image
Computational complexity	Very high; grows explosively with image resolution	Comparatively low; convolution + pooling keep computation manageable even for high-resolution images
Ability to handle image data	Poor — cannot exploit local correlations, prone to overfitting on high-dimensional image vectors	Excellent — naturally suited to images, learns hierarchical visual features automatically

Role of Convolutional Layers, Filters, and Pooling Layers

- Convolutional layers: Apply learnable filters over local regions of the X-ray image to produce feature maps highlighting edges, textures, and abnormal regions, instead of treating each pixel independently as an FCNN would.
- Filters: Small weight kernels that are shared/reused across the entire image; each filter specializes in detecting a specific local pattern (e.g., a bone edge or opacity boundary), giving the network translation invariance.
- Pooling layers: Downsample feature maps (e.g., max-pooling), retaining the strongest activations while reducing spatial size, computation, and sensitivity to small shifts or noise in the X-ray image.

Justification: Why CNNs Are More Suitable for Image Classification

CNNs preserve spatial structure, share parameters across the image (drastically fewer weights than an FCNN of comparable capacity), and automatically learn a hierarchy of features from simple edges to complex disease patterns. This makes them far more efficient, less overfitting-prone, and more accurate for image-based tasks such as X-ray classification than a fully connected network, which discards spatial information and requires an impractically large number of parameters.

Q2. Different CNN Layers and Filters (5 Marks)

Context: A manufacturing company is developing a CNN-based defect-detection system for industrial products.

Layer	Feature Extraction Role	Spatial Information	Computational Requirement	Role in Classification
Convolutional layer	Detects local visual patterns (edges, textures, defect boundaries) using learnable filters	Preserves spatial layout of the product image	Moderate — depends on filter size, number of filters, and stride	Provides the raw feature maps used by deeper layers
Pooling layer	Aggregates/summarizes nearby feature activations (e.g., max-pool)	Reduces spatial resolution while keeping dominant features	Low — simple downsampling operation, no learnable weights	Reduces featuremap size, controlling overfitting and computation before FC layers
Activation layer (ReLU)	Introduces non-linearity so the network can model complex, non-linear defect patterns	No spatial change — applied element-wise	Very low — simple thresholding operation	Enables the network to learn complex decision boundaries between defective/nondefective
Fully connected layer	Combines all high-level extracted features into a global representation	Spatial structure is discarded/flattene d at this stage	High — dense connections between all inputs and neurons	Produces the final defect classification decision

Early-Layer Filters vs. Deeper-Layer Filters

Aspect	Early-layer filters	Deeper-layer filters
Type of features learned	Low-level generic features: edges, corners, colour/intensity gradients, simple textures	High-level, task-specific features: defect shapes, scratch patterns, missingcomponent signatures
Generality	Fairly generic — similar across many vision tasks	Highly specific to the defect types present in the manufacturing dataset
Receptive field	Small, local receptive field	Large, effectively global receptive field built up from stacked convolutions
Sensitivity	Sensitive to raw pixel-level variations (noise, lighting)	Sensitive to abstract structural/semantic patterns rather than raw pixels

Justification: Collective Improvement of Defect Detection

Early convolutional layers extract basic visual primitives (edges, textures) common to all product images. Pooling layers compress this information while retaining the most salient activations and adding robustness to minor positional variation of defects. Activation layers allow the network to model the non-linear, irregular shapes that real defects take. Finally, fully connected layers integrate all these high-level features into a single decision. Together, this layered pipeline lets the system progress from raw pixels to a reliable, generalizable defect/nondefect classification.

Q3. Parameter Sharing vs Independent Weights (5 Marks)

Context: A smart surveillance system must process thousands of high-resolution images efficiently.

Aspect	Independent Weights (Fully Connected Network)	Parameter Sharing (CNN)
Number of parameters	Extremely large — one unique weight per input-output connection; grows quadratically with image size	Small — a filter's weights (e.g., 3×3×channels) are reused at every spatial location, independent of image size
Aspect	**Independent Weights (Fully Connected Network)**	**Parameter Sharing (CNN)**
Memory requirements	Very high memory footprint, especially for high-resolution surveillance frames	Much lower memory footprint since the same filter weights are stored once and reused
Computational complexity	High — every neuron requires a separate multiply-add for every input pixel	Lower relative to capacity — convolution operations are efficient and hardwareoptimized (e.g., GPU convolution kernels)
Feature detection	Learns a separate, unrelated feature detector for every position — inefficient and redundant	Learns one general-purpose detector per feature type, applied uniformly across the whole frame
Translation-related invariance	None — a pattern learned in one part of the image is not recognized if it appears elsewhere	Strong — the same filter detects a given pattern (e.g., a person's silhouette) wherever it appears in the frame

How Reuse of Filter Weights Improves Learning Efficiency

Because a single filter's weights are convolved across every location in the image, the network needs to learn a pattern (e.g., a face or vehicle edge) only once rather than separately for every possible pixel position. This reuse means far fewer parameters must be learned and stored, training converges faster with less data, and the same detector automatically generalizes to objects appearing anywhere in the frame — precisely the behaviour needed when people or vehicles move across a surveillance camera's field of view.

Justification for Large-Scale Image Processing Applications

For a system processing thousands of high-resolution images, independent weights would be computationally and memory-prohibitive, and would fail to generalize to objects at new positions. Parameter sharing keeps the model compact, memory-efficient, and translation-invariant, enabling real-time or near-real-time processing of large volumes of surveillance footage — making it essential for practical, large-scale deployment.

Q4. AlexNet vs ResNet (5 Marks)

Context: A computer vision company is choosing between AlexNet and ResNet for an image classification system.

Aspect	AlexNet	ResNet
Network depth	8 learned layers (5 convolutional + 3 fully connected)	Very deep — 18/34/50/101/152 layers, built from residual blocks
Layer organization	Sequential stack of conv, ReLU, max-pool, and large FC layers	Stacked residual blocks with identity/skip (shortcut) connections around groups of conv layers
Convolutional filters	Large filters in early layers (e.g., 11×11, 5×5), fewer total conv layers	Small, uniform filters (mostly 3×3 and 1×1 bottleneck filters) repeated across many blocks

Parameter requirements	~60 million parameters, concentrated heavily in the FC layers	Comparatively parameter-efficient per layer of depth; uses global average pooling instead of large FC layers
Feature extraction capability	Learns good low/mid-level features; limited high-level abstraction due to shallow depth	Learns rich, hierarchical, high-level features enabled by much greater depth
Training difficulty	Relatively straightforward to train given its shallow depth	Deep plain networks are hard to train, but residual connections make very deep ResNets stable and trainable
Vanishing-gradient problem	Becomes a serious issue if the network is made deeper than its original design	Directly addressed by skip connections, which let gradients flow unimpeded
Aspect	**AlexNet**	**ResNet**
		through the network
Computational cost	Lower FLOPs, faster inference, smaller memory footprint	Higher FLOPs and computation, though efficient variants (ResNet-18/34) narrow this gap
Classification performance	Good baseline accuracy; outperformed by deeper, modern architectures	State-of-the-art accuracy on complex, largescale, fine-grained image recognition benchmarks

Recommendation and Justification

Recommendation: ResNet is more suitable for a complex image classification application. Its residual connections allow it to be trained much deeper than AlexNet without suffering from vanishing gradients, giving it substantially stronger feature-learning capability and higher classification accuracy on complex, real-world datasets. Although ResNet incurs a higher computational cost, this is an acceptable trade-off for a computer vision company prioritizing accuracy, and efficient ResNet variants (e.g., ResNet-18/34) can balance performance with computational cost.

Q5. Regularized CNN vs Non-Regularized CNN (5 Marks)

Context: An agricultural organization's plant-disease classification CNN shows very high training accuracy but poor performance on unseen leaf images — a classic sign of overfitting.

Aspect	CNN Without Regularization	CNN With Dropout + Data Augmentation + Batch Normalization
Overfitting	High — the model memorizes training images (including noise/background), leading to a large train–test accuracy gap	Significantly reduced — regularization actively discourages memorization of training-specific detail
Generalization	Poor generalization to new, unseen leaf images with different lighting, angles, or backgrounds	Strong generalization; performs consistently well on unseen field images
Training stability	Can suffer from unstable gradients and internal covariate shift as training progresses	Batch normalization stabilizes layer inputs, allowing smoother, faster, more stable convergence
Computational requirements	Slightly lower per-epoch cost (no extra regularization overhead)	Marginally higher per-epoch cost, but far fewer wasted training epochs due to more stable convergence
Model performance (realworld)	High training accuracy but unreliable, poor real-world diagnostic performance	Balanced training/validation accuracy and reliable real-world disease-classification performance

Which Regularization Techniques Should Be Applied

- Data augmentation: Apply rotations, flips, brightness/contrast jitter, and slight zoom/crop to leaf images, simulating the natural variation of field conditions (lighting, angle, growth stage) and effectively enlarging the training set.
- Dropout: Randomly deactivate neurons in the fully connected layers during training so the model cannot rely on a narrow set of features (e.g., a specific leaf background), forcing it to learn more robust, diseaserelevant patterns.
- Batch normalization: Normalize activations within each mini-batch to stabilize and accelerate training, and provide a mild additional regularization effect.

Justification for Reliable Real-World Disease Classification

Since the un-regularized model already shows a clear overfitting symptom (high training accuracy, poor test accuracy), the combination of data augmentation (to expose the model to realistic field variation), dropout (to prevent over-reliance on specific features), and batch normalization (to stabilize training) directly targets this problem. Together, these techniques are the most appropriate approach for building a plant-disease classifier that generalizes reliably to new, unseen leaf images captured under real agricultural field conditions.